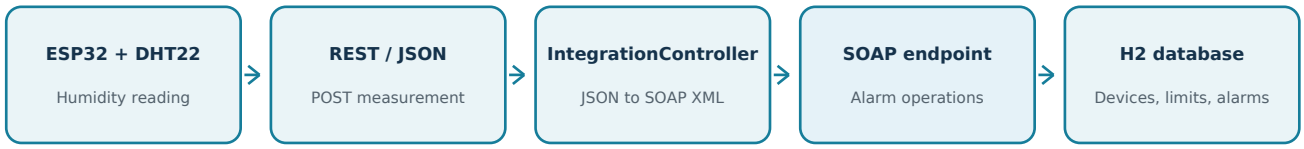
Individuell IoT-larmtjänst

Teknisk dokumentation | ESP32, REST/JSON och Java SOAP

Status: Java-tjänstens REST/SOAP-flöde har provats lokalt med Postman och SoapUI. Den fysiska ESP32-enheten har ännu inte verifierats. Tjänsten använder ett eget WSDL/XSD; kompatibilitet med ett gemensamt kurskontrakt har inte kunnat kontrolleras eftersom inget sådant kontrakt hittades i tillgängligt kursmaterial.

1. Syfte och arkitektur

Lösningen demonstrerar ett IoT-flöde där en ESP32 med en DHT22-sensor läser luftfuktighet. Enheten skickar JSON via HTTP till en REST-ingång. Java-applikationens IntegrationController kontrollerar och omvandlar mätningen till SOAP XML och anropar SOAP-tjänsten. AlarmService jämför värdet med en sparad gräns och sparar ett larm när gränsen överskrids.



I den fristående demonstrationen ligger REST-ingången och SOAP-tjänsten i samma Spring Boot-applikation på port 8083. IntegrationController sköter protokollöversättningen; larmbeslutet tas i AlarmService.

2. Teknik och ansvar

Del	Ansvar
ESP32 + DHT22	Läser luftfuktighet på GPIO4, ansluter till Wi-Fi och skickar mätvärden som JSON ungefär var 30:e sekund.
REST / IntegrationController	Tar emot enhetsregistrering och mätning, kontrollerar fält och registreringsstatus samt anropar SOAP med WebServiceTemplate.
AlarmEndpoint + AlarmService	Tar emot SOAP-operationer och avgör om mätningen passerar den konfigurerade gränsen.
Spring Data JPA + H2	Lagrar enheter, tröskelvärden och skapade larm lokalt.

3. Gränssnitt och funktioner

REST tar emot JSON och använder HTTP-statuskoder för att signalera resultat. Exempel på mätning:

Fält	Exempel
deviceId	Hum-01
measurementType	humidity
value / unit	68.2 / %
timestamp	2026-10-25T12:30:45Z

REST-operationer

• POST /api/devices registrerar en enhet med ID, mättyp och enhet. Enheten måste vara registrerad innan mätningar accepteras.

• POST /api/measurements validerar JSON, skickar mätningen vidare via SOAP och returnerar om ett larm skapades.

SOAP-operationer

• ProcessMeasurement bedömer mätningen mot rätt tröskel och skapar ett larm om värdet är högre än gränsen.

• GetAlarm hämtar ett larm via dess ID.

• GetAllAlarms returnerar alla sparade larm.

SOAP-kontraktet har namespace **urn:simon:iot:alarm:v1**. WSDL publiceras lokalt på **http://localhost:8083/ws/alarm.wsdl**; SOAP endpoint är **http://localhost:8083/ws**. XML-strukturer definieras i alarm.xsd.

4. Databas, gränsvärden och fel

H2-databasen lagrar enheter, tröskelregler och larm. Standardgränserna är luftfuktighet 60 % och temperatur 30 °C. Tröskelregler används av larmtjänsten vid bedömningen.

Mätning	Standardgräns	Utfall vid överskridande
Luftfuktighet	60 %	Larm skapas och sparas
Temperatur	30 °C	Larm skapas och sparas

REST svarar bland annat med 400 vid ogiltiga fält, 403 om enheten inte är registrerad och 502 om SOAP-anropet misslyckas. SOAP-fel används för ogiltiga SOAP-anrop och när tröskelregel saknas.

5. Hårdvara och uppkoppling

DHT22-signal	ESP32-anslutning
+	3V3
OUT	GPIO4 / D4
-	GND

Firmware använder Wi-Fi-uppgifter i arduino_secrets.h, som ska hållas utanför GitHub. Enhet och dator behöver vara på samma lokala nätverk; firmware ska riktas mot datorns lokala IP-adress och port 8083.

6. Start och testning

Java-tjänsten kräver JDK 21. Öppna alarm-service i IntelliJ och starta AlarmServiceApplication, eller kör **mvn spring-boot:run** i projektmappen. Tjänsten lyssnar på port 8083.

Genomförda lokala tester

• Start av Spring Boot-applikationen i IntelliJ.

• Postman: registrering av Hum-01 via POST /api/devices.

• Postman: mätning 58.4 % gav accepted=true och alarmCreated=false.

• Postman: mätning 68.2 % gav accepted=true och alarmCreated=true med alarm-ID 1.

• SoapUI: GetAlarm med ID 1 returnerade larmet.

• SoapUI: GetAllAlarms returnerade den sparade larmlistan.

Dessa resultat verifierar den lokala REST/SOAP-integrationen med testanrop. De verifierar inte att den fysiska ESP32-enheten kan ansluta eller skicka mätvärden.

7. Begränsningar och fortsatt arbete

• ESP32-firmware har inte laddats upp och körts tillsammans med DHT22; fysisk sensorläsning och sändning är därför inte verifierade.

• Nätverksorsak: datorn var ansluten med Ethernet direkt till fastighetsnätet och ingen lokal router fanns tillgänglig. ipconfig visade en offentlig adress på Ethernet, inte en lokal adress på ett gemensamt nätverk med ESP32. Adressen användes inte i firmware. En tillfällig lokal åtkomstpunkt/router behöver konfigureras för att ESP32 säkert ska kunna nå Java-tjänsten.

• Tillgängligt kursmaterial innehöll inget gemensamt WSDL/XSD eller REST-kontrakt. Projektet använder därför ett eget, fristående SOAP-kontrakt; kompatibilitet med ett eventuellt kurskontrakt är inte verifierad.

• ESP32 skickar till den lokala Spring Boot-tjänsten. Koppling till en separat grupp-Integration Service har inte demonstrerats.

• Fortsatt arbete: få lärarens besked om kontraktskrav, jämför WSDL och REST-format om de tillhandahålls, konfigurera den faktiska integrationsadressen och demonstrera hårdvaran.

8. Transparens om framtagandet

AI-stöd har använts för att ta fram kod- och dokumentationsutkast. Applikationen har därefter startats och REST/SOAP-anrop har provats av studenten med IntelliJ, Postman och SoapUI. Studenten bör följa kursens regler för redovisning av AI-stöd och kunna förklara de delar som lämnas in.

Projektfiler: alarm-service/ (Java REST/SOAP, WSDL/XSD och H2), producer-esp32/ (ESP32-firmware och exempelkonfiguration), demo/ (exempelpayloads) samt README.md.